

Advanced Topics in Operating Systems

Homework 1

Maha Zainab

February 2026

Question 1

1.1 CPU Utilization vs. Response Time

CPU utilization measures how busy the CPU is and response time measures how quickly the system produces the first response after a request arrives.

These two goals can conflict because improving response time often requires more frequent preemption, which increases overhead. For example, in Round Robin with a very small time quantum, interactive processes get CPU time quickly (good response time), but the CPU spends more time doing context switches and less time executing actual work, which can reduce CPU utilization.

On the other hand, if we try to maximize CPU utilization, we may use longer time slices or favor long CPU bursts to reduce context switching. This keeps the CPU busy with fewer interruptions (high utilization), but short/interactive jobs may wait longer before getting CPU time, which hurts response time.

So in many settings, especially time-sharing systems, there is a trade-off: aggressive preemption improves response time but can reduce CPU utilization, while minimizing overhead improves CPU utilization but can increase response time.

1.2 Average Turnaround Time vs. Maximum Waiting Time

Turnaround time is the total time from arrival to completion, and waiting time is the time spent in the ready queue. Minimizing average turnaround time can conflict with minimizing the maximum waiting time because an algorithm that optimizes the average may treat some processes very unfairly.

A classic example is Shortest Job First (SJF). SJF tends to minimize the average turnaround time by running short jobs early, which lowers the average completion time across all processes. However, this can make long jobs wait a very long time. In the worst case, if short jobs keep arriving, a long job may be postponed repeatedly, leading to starvation. Even if starvation does not happen, the maximum waiting time can become very large.

In contrast, algorithms designed to reduce the maximum waiting time (e.g., Round Robin or aging-based priority schemes) may sometimes run longer jobs earlier than SJF would, which can increase the average turnaround time even though it improves fairness.

Therefore, reducing average turnaround time often comes at the cost of increasing the maximum waiting time for some processes, and vice versa.

Question 2

All processes arrive at time 0 in the order P_1, P_2, P_3, P_4, P_5 .

Given Process Set

Process	Burst Time (ms)	Priority
P_1	2	2
P_2	1	1
P_3	8	4
P_4	4	2
P_5	5	3

2.1 Gantt Charts

(a) FCFS

Order: $P_1 \rightarrow P_2 \rightarrow P_3 \rightarrow P_4 \rightarrow P_5$

```
0    2    3        11        15        20
| P1 | P2 |   P3   |   P4   |   P5   |
```

(b) SJF (Non-preemptive)

Order by shortest burst: $P_2(1), P_1(2), P_4(4), P_5(5), P_3(8)$

```
0    1    3        7        12        20
|P2 | P1 |   P4   |   P5   |   P3   |
```

(c) Non-preemptive Priority

Higher number = higher priority. Order: $P_3(4), P_5(3), P_1(2), P_4(2), P_2(1)$. (Tie between P_1 and P_4 broken by arrival order.)

```
0          8        13   15        19   20
|    P3    | P5 | P1 | P4 | P2 |
```

(d) Round Robin (Quantum = 2)

```
0  2  3  5  7  9   11  13  15  17  18  20
|P1|P2|P3|P4|P5| P3 | P4 | P5 | P3 | P5 | P3 |
```

Another way to represent Gantt Chart.

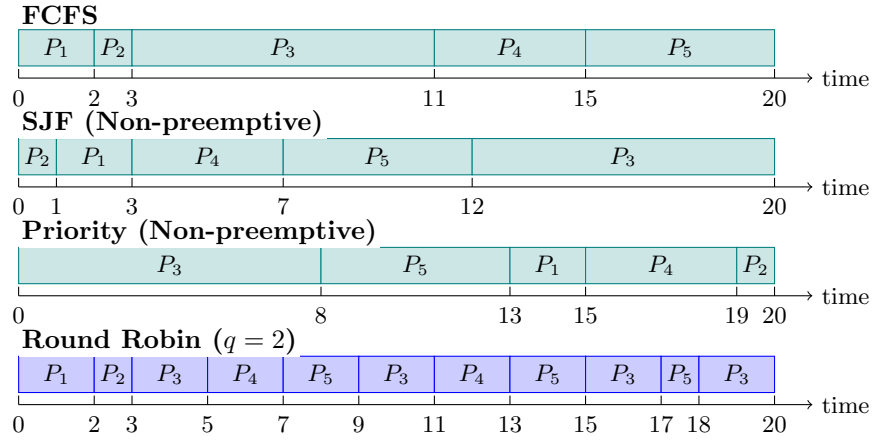


Figure 1: Gantt charts for FCFS, SJF (non-preemptive), non-preemptive Priority, and Round Robin ($q = 2$).

2.2 Turnaround Time

As we know, all arrival times are 0, turnaround time = completion time.

Process	FCFS	SJF	Priority	RR(q=2)
P_1	2	3	15	2
P_2	3	1	20	3
P_3	11	20	8	20
P_4	15	7	19	13
P_5	20	12	13	18

2.3 Waiting Time

Waiting time = Turnaround time – Burst time.

Process	FCFS	SJF	Priority	RR(q=2)
P_1	0	1	13	0
P_2	2	0	19	2
P_3	3	12	0	12
P_4	11	3	15	9
P_5	15	7	8	13

2.4 Minimum Average Waiting Time

$$\text{FCFS avg} = \frac{0 + 2 + 3 + 11 + 15}{5} = 6.2$$

$$\text{SJF avg} = \frac{1 + 0 + 12 + 3 + 7}{5} = 4.6$$

$$\text{Priority avg} = \frac{13 + 19 + 0 + 15 + 8}{5} = 11.0$$

$$\text{RR avg} = \frac{0 + 2 + 12 + 9 + 13}{5} = 7.2$$

By comparing above calculations, we can say that SJF gives the minimum average waiting time.

Question 3

Which of the following scheduling algorithms could result in starvation? Why?

Starvation happens when a process waits for the CPU for an unbounded time because other processes keep getting scheduled ahead of it.

1. **First-Come, First-Served (FCFS):** FCFS does not cause starvation because it runs processes strictly in arrival order, so every process will eventually run. A process may wait a long time if it is behind a long CPU burst, but it will not be postponed forever.
2. **Shortest Job First (SJF):** SJF can cause starvation because it always prefers the shortest CPU burst. If short jobs keep arriving, a long job can keep getting pushed back and may wait indefinitely.
3. **Round Robin (RR):** RR does not cause starvation, it cycles through the ready queue, giving each process a fixed time quantum. As long as the ready queue is managed fairly, every process gets CPU time regularly, so starvation does not occur.
4. **Priority Scheduling:** Priority scheduling can cause starvation with strict priorities, low-priority processes may never execute if higher-priority processes continue to arrive. This can be fixed using aging, where the priority of waiting processes increases over time.

Question 4

We are given a preemptive priority scheduling algorithm where:

- Larger priority numbers mean higher priority.
- When a process is **waiting**, its priority increases at rate a .
- When a process is **running**, its priority increases at rate b .
- All processes enter the ready queue with priority 0.

We are asked to determine what scheduling algorithm results when:

$$b > a > 0$$

Analysis

As $b > a$, the priority of the running process increases faster than the priority of waiting processes.

This means that once a process starts running, its priority keeps increasing faster than all other processes. Waiting processes cannot catch up in priority. Therefore, the running process will not be preempted.

Even though the algorithm is technically preemptive, in practice no preemption will occur once a process begins execution.

Because all processes start with priority 0, the first process selected is determined by arrival order. Since all processes arrive at time 0, the tie is broken by their order in the ready queue.

Thus, each process runs to completion before the next one starts.

When $b > a > 0$, the system behaves like First-Come, First-Served (FCFS) because once a process begins execution, its priority increases faster than others and it continues running until completion. As the running process's priority grows faster, it remains highest priority, so no other process can preempt it.